

Vaada: presenter's script

The 15 minutes you drive. Six minutes of product, nine of system design, then they take over with questions.

This is close to word for word. Don't memorise it. Read it three times, do one full run with a timer, and then present from the timing table on the last page. If you try to recite this verbatim you'll sound like you're reciting it, which is worse than being a bit loose.

Bold lines are the ones that must land. Everything else can be paraphrased, shortened, or dropped. If you're running out of air, say the bold line and move on.

Before you start

Ten minutes before the call:

- Sign in to <https://vaada-crm.vercel.app> already. Don't type a password on a shared screen.
- Open `system-design-deck.html` in a second tab, full screen. Arrow keys move it.
- Open the editor on `src/lib/ai/gemini.ts`, in case they want real code mid-flow.
- Hit <https://vaada-crm.vercel.app/api/health> and confirm `status: ok`. The first request after idle can take a second because the database is cold.
- If the demo data looks messy from earlier clicking, run `npm run db:seed` to reset it to one lead per stage.
- Close Slack, mail, and notifications.
- Share the browser window, not the whole desktop.

Have water. You're talking for fifteen minutes straight.

Opening (20 seconds)

Don't start by clicking. Start by framing, or they'll spend the first minute working out what they're looking at.

"This is Vaada, a lead CRM for small sales teams in India. The name means promise.

The bet behind it is that for a small team, the expensive failure isn't a missing dashboard. It's telling a customer you'll call them Tuesday and then not calling. That's lost trust and lost revenue.

So the whole product is organised around promises, and everything you're about to see comes back to that. I'll do about six minutes on the product, then spend the rest on how it's built."

Then share the screen and go.

Part 1a — Product demo (6 minutes)

Dashboard (55s)

[Start on `/dashboard` .]

"This is where you land.

Most CRMs open on charts. This one opens on what you've already promised people.

The attention queue is everything overdue or due today. Overdue first, then today's, and inside each group the latest ones sort to the top.

On the right, pipeline shape and recent activity. Those are real numbers with nowhere else to live.

One thing worth mentioning: I had a metrics strip across the top and I cut it. Two of the four cards were re-showing counts the queue already showed one section down. **It looked like a dashboard, but it wasn't telling you anything new.**"

[Point at the coloured dots as you say it.]

Leads and pipeline (80s)

[Click Leads.]

"Leads. Searchable, filterable, archivable."

[Type into search, then clear it.]

"Every one of these queries filters by who's logged in, and that filter is inside the database query, not a permission check sitting above it. I'll come back to why that distinction matters in the design half, because it's the thing I'd most want you to look at."

[Click Pipeline. Drag a card one column across.]

"Same lead data, laid out by stage. Cards drag between stages and it writes through immediately. **Every card also has a stage dropdown, so dragging is never the only way to do it.** That's a keyboard and screen reader path, not a nice-to-have."

Lead detail (45s)

[Open any lead.]

"A single lead. Contact details, notes, stage, and the full follow-up history underneath."

[Edit a field. Save.]

"Saves use optimistic concurrency. If someone else edited this lead between me opening it and me saving, I get a conflict back instead of silently overwriting their work."

It's a small thing for a demo with one user. It's not a small thing for a sales team sharing accounts."

The AI, both lead-level features (110s)

This is the section they care about most. Slow down here.

[Scroll to the AI panel. Click Lead insight.]

"Now the AI. **Every Gemini call runs on the server. The API key never reaches the browser.**

While that's running, worth saying what's actually being sent. **No phone number, no email, no full name.** Stage, industry, city, source, the notes, recent follow-ups. And the deal value goes as a band, under fifty thousand, fifty to two-fifty, over two-fifty, rather than the exact number.

That's a boundary in code, not an instruction in the prompt. If a field never goes into the request it can't come back out."

[Insight appears.]

"And it comes back structured. Opportunity, risk, the evidence it used, a recommended next action, a confidence level, and a caveat.

It has to come back as JSON matching a schema, and I validate it again on my side regardless of what the model says it returned. Being told to return a shape isn't the same as getting one."

[Click WhatsApp draft. Wait. Then edit a word in the textarea.]

"Second feature drafts an outreach message. And it's editable, which is the point."

[Type into it.]

"Nothing here sends anything. There is no send button. The AI writes a starting point, a person decides. Same with saving, that's a deliberate click every time."

Follow-ups and the daily brief (70s)

[Click Follow-ups. Complete one.]

"Follow-ups are classified in India time, not the server's UTC, which matters at day boundaries.

Completing one stamps a completion time. **It doesn't move the lead's stage and it doesn't message anybody.** The system doesn't guess what you meant."

[Click AI brief. Generate.]

"Third AI feature, a daily brief across the active leads."

[When it lands, dismiss one priority.]

"Ranked priorities, risks, and what's going well. I can edit the summary, and I can drop anything I don't care about before saving it.

And every lead it references gets checked against the database before it can be saved. If the model invents an ID, or names a lead belonging to someone else, it's rejected. **You don't stop a model hallucinating. You make hallucinating harmless."**

Transition (10s)

Check the clock. If you're past seven minutes, stop demoing now regardless of what you haven't shown.

"That's the product. Let me show you how it's put together."

[Switch to the deck tab.]

Part 1b — System design (9 minutes)

Fifteen slides. Times below are maximums, not targets.

Slide 1 — Title (10s)

"Three things I want to cover: how it's structured, how the AI layer stays safe, and what I'd do differently with more time."

Slide 2 — Architecture (60s)

"One Next.js app. Pages, API routes, and the Gemini calls all deploy together. Postgres behind it, on Neon. Gemini called only from the server.

I deliberately didn't split a separate backend out. For one developer, free tier, under five requests a second, that would have meant two deployments, duplicated config, and a CORS surface to manage, and I'd have got nothing for it.

What I did instead was keep the seams clean. Domain logic, AI logic and routes are separate modules, so the API could be pulled out later without a rewrite. If this had real load or more than one team, I'd split it. Until something measured says so, it's operational cost for no benefit."

Slide 3 — Auth (35s)

"Signing in. Email and password, validated, then bcrypt compares against the stored hash. **bcrypt is deliberately slow, which is the whole point.** If the database leaks, an attacker gets thousands of guesses a second instead of billions.

Success gives you a signed HttpOnly cookie, so JavaScript on the page can't read it.

The trade-off is that I can't force a logout before the token expires, because there's no session table. Eight hour expiry. For production I'd want OIDC or verified magic links, MFA, and session revocation."

Slide 4 — Data model (30s)

"Two of the five tables exist purely for the AI.

One stores results a user chose to keep. The other logs **every** call, including the ones that failed.

They're separate on purpose. Merge them and you either store results nobody wanted, or you lose the record of calls that failed before producing anything. And I need those, both to debug and because that table is what the rate limiter counts."

Slide 5 — Owner scoping (65s)

This is your strongest slide. Give it the time.

"This is the bit I'd point at first if you only looked at one thing."

Every query filters by owner ID, and that filter is inside the database call.

Here's why that phrasing matters. Suppose I only checked 'is this person logged in' at the top of the handler, then fetched the lead by ID. I sign in as myself, then ask for your lead's ID. I'm logged in, so the check passes, and the database hands me your data. **That's a cross-customer breach from one missing filter.** It's called an IDOR and it's one of the most common serious web vulnerabilities.

With the owner in the where clause, asking for someone else's record returns nothing. It 404s. You can't even confirm it exists.

The second half of this slide is the conflict handling. The client sends back the `updatedAt` it last saw, and that goes into the where clause too. If someone else saved first, zero rows match, and then a follow-up count tells me whether it's a conflict or genuinely deleted. 409 or 404.

I used `updatedAt` rather than adding a version column because Prisma already maintains it. Same guarantee, one less field."

Slide 6 — What Gemini receives (50s)

"This is the actual function that builds the context.

No phone, no email, no full name. The value becomes a band. Notes get control characters stripped and cut to six hundred characters. Only the message draft adds anything, and only a first name, because a message addressed to nobody is useless.

The reason it's a function and not a line in the prompt is that prompts are requests and code is a guarantee. I'd rather the data never enter the request than ask a model nicely not to repeat it."

Slide 7 — Structured output (35s)

"Every response has a contract like this one.

I generate a JSON schema from it, send that to Gemini, and then **when the response comes back I validate it against the same schema again.**

That double-check catches malformed JSON, missing fields, and the occasional response that just ignores the schema. Models are non-deterministic. Anything I don't verify, I don't actually know."

Slide 8 — Guards before the call (35s)

"Two things happen before any request goes out.

A rate limit, five per user per rolling minute, **counted in Postgres rather than in memory.** Serverless instances don't share memory, so an in-process counter would let the real limit be five times however many instances happen to be warm.

Then I write a row before the call, marked started. **If the process dies mid-request there's still a record that it happened.**"

Slide 9 — Retry (45s)

"When it fails, the failure gets categorised.

Only timeouts, 5xx and rate limits get the one retry. A malformed request or a safety block will fail identically the second time, so retrying just spends the timeout budget on a guaranteed failure. The retry has random jitter on it, so that if lots of clients fail at once they don't all come back at the same instant and knock over a service that's already struggling. And every attempt is logged with its outcome, duration and retry count. **No prompt content and no PII in the logs.**"

Slide 10 — Fallbacks (30s)

"If Gemini is unavailable, or the key isn't configured, or you're rate limited, these run instead. Built from CRM data only, and labelled in the UI so nobody thinks they're getting AI output when they aren't."

Slide 11 — The fallback test (20s)

"And there's a test holding them to the same schema a real response has to satisfy. **So the UI genuinely can't tell them apart, and there's no second rendering path to maintain.** The product stays useful with zero AI budget."

Slide 12 — Deploy and monitoring (45s)

"One Vercel deploy, one Neon database, both free tier. Migrations are deployable and the seed only touches the demo user. There's a public health endpoint that actually pings the database and reports the deployed version and latency. GitHub Actions hits it every five minutes and **fails the run unless both the app and the database report ok.** It parses the body rather than just checking for a 200, because an app can happily return 200 while its database is unreachable. Run history is public. Structured JSON logs to Vercel, and every AI call's outcome and duration is queryable in Postgres."

Slide 13 — Quality and the honest bit (50s)

Do not skip this and do not rush it. Volunteering this is worth more than the coverage number costs you.

"On testing, here's the honest version. Domain and AI logic sit at ninety-five percent branch coverage. That's where a bug is silent and expensive, things like date bucketing marking something as not-overdue when it is. **Across the whole project it's 6.34 percent, because routes and components have no unit tests at all.** I verified those by hand against the deployment. **That's short of the target I set myself, and it's written down in DECISIONS.md rather than rounded up.** If I had another day, route tests are the highest-value next thing, because they're the actual contract boundary and they mock easily."

Same honesty on Lighthouse. Ninety-nine on performance, but on the login page only, because the CLI can't carry a session through the login. The signed-in pages should be similar since they share the build, but I haven't measured them, so I won't claim it."

Slide 14 — Scale (40s)

"If this had to carry real load:

A tenant model with row-level security in Postgres, **so isolation is enforced by the database rather than by me remembering to filter every query.** That's the big one.

Move the AI work onto a queue so it's off the request path. Rate limiting to Redis. Read replicas or materialized aggregates for the dashboard. Real tracing and cost metrics, plus prompt regression evals so a prompt change doesn't quietly make the output worse.

Splitting into services comes last, and only when something measured says so."

Slide 15 — Close (10s)

"That's Vaada. Happy to go deeper on any of it. The trade-offs are all written up in DECISIONS.md, including the parts I didn't finish."

Then stop talking. Let them ask.

When things go wrong

They interrupt with a question mid-demo

Normal and good. It means they're engaged.

Answer briefly, then steer back:

"Good question. Short answer is [one or two sentences]. I go into that properly in a couple of slides, want me to keep going and come back to it?"

Don't get pulled into a five-minute tangent at minute three. You'll run out of time and never reach the design half, which is the half being graded hardest.

Gemini fails or rate limits during the demo

This is a gift. Take it.

"There we go, that's the fallback firing. Which is actually convenient, because that's a designed behaviour rather than an error. It's rules-based, built from CRM data, it satisfies the same schema a real response does, and it's labelled so nobody thinks it's AI output. **The product stays usable with the AI completely unavailable.**"

Being calm about a dependency failing, in a project built around graceful degradation, is a better demonstration than the happy path.

The deployment is down

Say it plainly, don't fumble:

"Looks like the deployment isn't responding. Rather than debug it live, I've got a recording of the product walkthrough. Let me run that, and I'll come back to the live app afterwards if it recovers."

`vaada-demo.mp4` is on your machine, first 3.5 minutes is the product half.

You blank

Stop. Breathe. Say:

"Let me come back to that."

Move to the next slide. Nobody remembers this ten minutes later. Filling silence with waffle is far more damaging than a two-second pause.

You're running long

Check the clock at 6 minutes and 11 minutes. If you're behind, cut in this order and say you're cutting rather than dropping it silently:

1. The daily brief in the demo, it's the least surprising of the three
2. Slide 11, the fallback test
3. Slide 4, the data model

Never cut slide 5 (owner scoping), the Gemini block, or slide 13 (honesty). Those three earn the most.

Timing card

Present from this, not the full script.

Time	What
0:00	Framing. Promise, not dashboards
0:20	Dashboard. Attention queue, cut the metrics strip
1:15	Leads, search, pipeline, drag. Owner filter is in the query
2:35	Lead detail. Optimistic concurrency
3:20	Lead insight. No PII, value as a band, schema-validated
4:20	WhatsApp draft. Edit it. Nothing sends
5:10	Follow-ups, complete one. Daily brief, dismiss one, IDs verified
6:20	Switch to deck
6:30	Architecture. One app, why not split, clean seams
7:30	Auth. bcrypt slow on purpose, can't force logout
8:05	Data model. Two AI tables, why separate
8:35	Owner scoping. IDOR. 409 vs 404
9:40	Context. Boundary in code, not a prompt
10:30	Schemas. Validate again regardless
11:05	Guards. DB rate limit, row before the call
11:40	Retry. Only what's worth retrying, jitter
12:25	Fallbacks, and the test
13:15	Deploy and monitoring. Parses the body
14:00	Quality. Say 6.34% yourself
14:50	Scale. RLS first, services last
15:00	Close, stop talking

If you rehearse one thing

Slide 5, owner scoping. Say it out loud until it's smooth:

"Every query filters by owner ID, inside the database call. If it were a permission check above the query instead, I could sign in as myself, ask for your lead by ID, pass the logged-in check, and get your data back. That's an IDOR. With the owner in the where clause it returns nothing and 404s."

If that comes out clean and unhesitating, you sound like someone who understands their own security model. That single answer moves the needle more than any other thirty seconds in the fifteen minutes.